BÀI TẬP THỰC HÀNH
MÁY HỌC VỚI PYSPARK

1. Mục tiêu thực hành

- Hiểu vai trò của Apache Spark trong xử lý Machine Learning trên dữ liệu lớn.
- Đọc và chuẩn bị dữ liệu bằng Spark DataFrame.
- Mã hóa biến phân loại bằng StringIndexer và OneHotEncoder.
- Ghép các predictor thành cột features bằng VectorAssembler.
- Xây dựng mô hình Decision Tree và Logistic Regression cho bài toán phân loại.
- Đánh giá classifier bằng confusion matrix, Accuracy, Precision, Recall, F1 và AUC.
- Biến văn bản thành vector bằng Tokenizer, StopWordsRemover, HashingTF và IDF.
- Xây dựng Linear Regression và đánh giá bằng RMSE, MAE, MSE, R^2.
- Thực hiện feature engineering, bucketing, Ridge/Lasso/Elastic Net.
- Xây dựng Pipeline, tránh data leakage và tuning bằng Cross Validation/Grid Search.
- Làm quen với ensemble: Random Forest và Gradient-Boosted Trees.

2. Chuẩn bị

- Python 3.x và Jupyter Notebook/JupyterLab hoặc VS Code.
- Java tương thích với PySpark.
- PySpark và các thư viện Python cần thiết.
- Notebook Machine_Learning_with_PySpark.ipynb.
- Thư mục data chứa flights-larger.csv và sms.csv.

```
pip install pyspark pandas matplotlib
```

3. Nội dung thực hành

Machine Learning học quy luật từ dữ liệu thay vì yêu cầu lập trình viên mô tả toàn bộ quy tắc thủ công. Bài giảng phân biệt hai nhóm bài toán cơ bản: Regression dự đoán giá trị liên tục và Classification dự đoán nhãn/lớp.

Spark phù hợp khi dữ liệu lớn hơn khả năng xử lý thuận tiện trên một máy: dữ liệu có thể được phân tán trên cluster, xử lý song song và khai thác bộ nhớ. Trong bài thực hành sử dụng API DataFrame-based pyspark.ml.

Khởi tạo SparkSession

```
import pyspark
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import *

spark = (
    SparkSession.builder
    .master("local[*]")
    .appName("MachineLearningWithPySpark")
    .getOrCreate()
)

print("PySpark:", pyspark.__version__)
print("Spark  :", spark.version)
```

local[*] sử dụng các CPU core khả dụng trên máy cục bộ. Khi chạy trên cluster, master sẽ được cấu hình phù hợp với hệ thống triển khai.

1. Đọc và khảo sát dữ liệu

1.1. Dữ liệu chuyến bay

```
flights = spark.read.csv(
    "data/flights-larger.csv",
    header=True,
    inferSchema=True,
    nullValue="NA"
)

flights.show(5)
flights.printSchema()
print("Rows:", flights.count())
```

1.2. Các cột chính

Cột	Ý nghĩa
mon	Tháng
dom	Ngày trong tháng
dow	Ngày trong tuần
carrier	Hãng bay
flight	Số hiệu chuyến bay
org	Sân bay xuất phát
mile	Quãng đường
depart	Giờ khởi hành
duration	Thời gian bay
delay	Độ trễ

1.3. Dữ liệu SMS với schema tường minh

```
sms_schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("text", StringType(), True),
    StructField("label", IntegerType(), True)
])

sms = spark.read.csv(
    "data/sms.csv",
    sep=";",
    header=False,
    schema=sms_schema
)

sms.show(5, truncate=False)
```

2. Chuẩn bị dữ liệu cho Machine Learning

Quy trình chuẩn bị dữ liệu thường gồm: loại cột không cần thiết, xử lý missing values, tạo feature mới, tạo label và mã hóa biến phân loại.

2.1. Kiểm tra missing values

```
flights.select([
    F.sum(F.col(c).isNull().cast("int")).alias(c)
    for c in flights.columns
]).show()
```

2.2. Làm sạch và tạo label

```
flights_ml = (
    flights
    .drop("flight")
    .filter(F.col("delay").isNotNull())
    .dropna()
    .withColumn("km", F.round(F.col("mile") * 1.60934, 0))
    .drop("mile")
    .withColumn("label", (F.col("delay") >= 15).cast("integer"))
)
```

Trong bài toán phân loại chuyến bay: label = 1 nếu delay >= 15 phút; label = 0 nếu ngược lại.

3. StringIndexer, OneHotEncoder và VectorAssembler

3.1. StringIndexer

```
from pyspark.ml.feature import StringIndexer

carrier_indexer = StringIndexer(
    inputCol="carrier", outputCol="carrier_idx",
    handleInvalid="keep"
)
flights_idx = carrier_indexer.fit(flights_ml).transform(flights_ml)
```

StringIndexer chuyển category dạng chuỗi thành chỉ số số. Tuy nhiên, chỉ số này không nên luôn được xem như một đại lượng liên tục có thứ tự.

3.2. OneHotEncoder

```
from pyspark.ml.feature import OneHotEncoder

onehot = OneHotEncoder(
    inputCols=["org_idx"],
    outputCols=["org_dummy"],
    dropLast=True
)
```

One-Hot Encoding tạo vector nhị phân/sparse vector, tránh tạo quan hệ thứ tự giả giữa các category.

3.3. VectorAssembler

```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=["mon", "dom", "dow", "carrier_idx", "org_idx", "km", "depart", "duration"],
    outputCol="features"
)
```

```
flights_ready = assembler.transform(flights_idx)
```

3.4. Chia train/test

```
train_df, test_df = flights_ready.randomSplit([0.8, 0.2], seed=23)
print(train_df.count(), test_df.count())
```

4. Decision Tree Classification

Decision Tree liên tiếp chia dữ liệu tại các node. Mỗi split chọn một điều kiện giúp phân tách lớp tốt hơn; leaf node đưa ra dự đoán cuối cùng.

```
from pyspark.ml.classification import DecisionTreeClassifier

tree = DecisionTreeClassifier(
    labelCol="label",
    featuresCol="features",
    maxDepth=5,
    seed=23
)

tree_model = tree.fit(train_df)
tree_pred = tree_model.transform(test_df)
tree_pred.select("label", "prediction", "probability").show(10)
```

4.1. Xem cấu trúc cây

```
print(tree_model.toDebugString)
```

5. Logistic Regression và đánh giá classifier

```
from pyspark.ml.classification import LogisticRegression

logistic = LogisticRegression(
    labelCol="label",
    featuresCol="features",
    maxIter=50
)

log_model = logistic.fit(train_df)
log_pred = log_model.transform(test_df)
```

5.1. Confusion Matrix

```
log_pred.groupBy("label", "prediction").count().show()
```

Thành phần	Điều kiện	Ý nghĩa
TP	label=1, prediction=1	Dự đoán dương đúng
FP	label=0, prediction=1	Dự đoán dương sai
FN	label=1, prediction=0	Bỏ sót mẫu dương
TN	label=0, prediction=0	Dự đoán âm đúng

5.2. Accuracy, Precision, Recall, F1, AUC

```
from pyspark.ml.evaluation import (
    MulticlassClassificationEvaluator,
    BinaryClassificationEvaluator
)

multi = MulticlassClassificationEvaluator(labelCol="label")
binary = BinaryClassificationEvaluator(labelCol="label")

for metric in ["accuracy", "f1", "weightedPrecision", "weightedRecall"]:
    print(metric, multi.evaluate(log_pred, {multi.metricName: metric}))

print("AUC", binary.evaluate(log_pred))
```

Lưu ý: Không nên chỉ nhìn Accuracy. Với dữ liệu lệch lớp, cần xem thêm Precision, Recall, F1 và AUC.

6. Text Mining và SMS Spam Classification

Pipeline xử lý văn bản trong slide: làm sạch dấu câu -> Tokenizer -> StopWordsRemover -> HashingTF -> IDF -> features.

6.1. Làm sạch và tokenize

```
from pyspark.sql.functions import regexp_replace
from pyspark.ml.feature import Tokenizer, StopWordsRemover

sms_clean = sms.withColumn("text", regexp_replace("text", r"[_():;,.!?\-]", " "))
sms_tokens = Tokenizer(inputCol="text", outputCol="tokens").transform(sms_clean)
sms_words = StopWordsRemover(inputCol="tokens", outputCol="words").transform(sms_tokens)
```

6.2. HashingTF + IDF

```
from pyspark.ml.feature import HashingTF, IDF

sms_hash = HashingTF(
    inputCol="words", outputCol="hash", numFeatures=1024
).transform(sms_words)

idf_model = IDF(inputCol="hash", outputCol="features").fit(sms_hash)
sms_tfidf = idf_model.transform(sms_hash)
```

6.3. Logistic Regression cho SMS

```
sms_train, sms_test = sms_tfidf.select("label", "features").randomSplit([0.8, 0.2], seed=13)

sms_model = LogisticRegression(
    labelCol="label",
    featuresCol="features",
    regParam=0.2
).fit(sms_train)

sms_pred = sms_model.transform(sms_test)
```

7. Linear Regression

Trong phần Regression, mục tiêu là dự đoán một giá trị liên tục. Bài thực hành dùng quãng đường để dự đoán thời gian bay duration.

```
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator
```

```
reg_base = flights.filter("duration IS NOT NULL AND mile IS NOT NULL")     .withColumn("km",
F.round(F.col("mile") * 1.60934, 0))

reg_assembler = VectorAssembler(inputCols=["km"], outputCol="features")
reg_data = reg_assembler.transform(reg_base)
reg_train, reg_test = reg_data.randomSplit([0.8, 0.2], seed=42)

lr_model = LinearRegression(labelCol="duration").fit(reg_train)
reg_pred = lr_model.transform(reg_test)
```

7.1. Các metric regression

```
for metric in ["rmse", "mae", "mse", "r2"]:
    evaluator = RegressionEvaluator(
        labelCol="duration",
        predictionCol="prediction",
        metricName=metric
    )
    print(metric, evaluator.evaluate(reg_pred))
```

7.2. Intercept và coefficient

```
print("Intercept:", lr_model.intercept)
print("Coefficients:", lr_model.coefficients)
```

8. Feature Engineering và Bucketing

Feature engineering tạo biến mới nhằm biểu diễn dữ liệu phù hợp hơn với mô hình. Slide minh họa bucketing, log, sqrt, power, product và ratio.

8.1. Bucketizer cho giờ khởi hành

```
from pyspark.ml.feature import Bucketizer

bucketizer = Bucketizer(
    splits=[0, 6, 12, 18, 24],
    inputCol="depart",
    outputCol="depart_bucket",
    handleInvalid="keep"
)

bucketed = bucketizer.transform(flights.filter(F.col("depart").isNotNull()))
bucketed.groupBy("depart_bucket").count().show()
```

8.2. Feature công thức

```
engineered = flights     .filter("mile IS NOT NULL AND duration IS NOT NULL")     .withColumn("km",
F.col("mile") * 1.60934)     .withColumn("speed_km_per_min", F.col("km") / F.col("duration"))
.withColumn("sqrt_km", F.sqrt("km"))     .withColumn("log_km", F.log1p("km"))
```

9. Regularization: Ridge, Lasso, Elastic Net

Regularization bổ sung penalty lên hệ số để hạn chế overfitting và kiểm soát độ phức tạp của mô hình.

Mô hình	elasticNetParam	Ý nghĩa
Ridge	0	L2 penalty
Elastic Net	0 < alpha < 1	Kết hợp L1 và L2

Lasso	1	L1 penalty; có thể đưa hệ số về 0

```
for name, alpha in [("Ridge", 0.0), ("ElasticNet", 0.5), ("Lasso", 1.0)]:
    model = LinearRegression(
        labelCol="duration",
        featuresCol="features2",
        regParam=0.1,
        elasticNetParam=alpha
    ).fit(reg_train3)
```

10. Pipeline và Data Leakage

Các transformer/estimator cần học tham số phải fit trên training data. Nếu fit trên toàn bộ dữ liệu trước khi chia train/test, thông tin của test đã rò rỉ vào bước preprocessing.

FIT: chỉ TRAIN | TRANSFORM: TRAIN và TEST

```
from pyspark.ml import Pipeline

pipeline = Pipeline(stages=[
    pipe_indexer,
    pipe_onehot,
    pipe_assembler,
    pipe_lr
])

pipeline_model = pipeline.fit(pipe_train)
pipe_pred = pipeline_model.transform(pipe_test)
```

Lưu ý: Đây là lý do nên đặt preprocessing và model vào cùng Pipeline khi có bước cần fit như StringIndexer, OneHotEncoder hoặc IDF.

11. Cross Validation và Grid Search

Cross Validation chia training data thành nhiều fold để ước lượng hiệu năng ổn định hơn và lựa chọn hyperparameter. Test set phải được giữ riêng cho đánh giá cuối cùng.

```
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator

param_grid = (
    ParamGridBuilder()
    .addGrid(cv_regression.regParam, [0.01, 0.1, 1.0])
    .addGrid(cv_regression.elasticNetParam, [0.0, 0.5, 1.0])
    .build()
)

cv = CrossValidator(
    estimator=cv_pipeline,
    estimatorParamMaps=param_grid,
    evaluator=cv_evaluator,
    numFolds=3,
    seed=13,
    parallelism=2
)

cv_model = cv.fit(pipe_train)
```

11.1. Xem mô hình tốt nhất

```
best_lr = cv_model.bestModel.stages[-1]
print(best_lr.getRegParam())
print(best_lr.getElasticNetParam())
```

12. Ensemble Models

Ensemble kết hợp nhiều cây/mô hình để tăng khả năng khái quát. Phần mở rộng sử dụng Decision Tree, Random Forest và Gradient-Boosted Trees.

```
from pyspark.ml.classification import (
    DecisionTreeClassifier,
    RandomForestClassifier,
    GBTClassifier
)

models = {
    "Decision Tree": DecisionTreeClassifier(seed=17),
    "Random Forest": RandomForestClassifier(numTrees=30, seed=17),
    "GBT": GBTClassifier(maxIter=20, seed=17)
}
```

12.1. So sánh mô hình

Với mỗi mô hình, tính AUC và F1 trên cùng test set. Sau đó chọn mô hình dựa trên metric phù hợp mục tiêu bài toán, không chỉ dựa vào thời gian huấn luyện.

4. BÀI TẬP MỞ RỘNG

Project A - Flight Delay Classification

1. Làm sạch flights-larger.csv.
2. Tạo label: delay >= 15 phút.
3. Mã hóa carrier và org.
4. Tạo Pipeline preprocessing.
5. So sánh Logistic Regression, Decision Tree, Random Forest và GBT.
6. Báo cáo Accuracy, F1, AUC.
7. Tuning ít nhất một mô hình bằng Cross Validation.
8. Giải thích lý do chọn mô hình cuối cùng.

Project B - SMS Spam Classification

1. Làm sạch text và tokenize.
2. Loại stop words.
3. HashingTF + IDF.
4. Logistic Regression.
5. Thử numFeatures = 1024, 4096, 16384.
6. Thử regParam/elasticNetParam.
7. So sánh F1 và AUC.

5. BÀI TẬP THÊM

Bài 1 - Data Preparation

- Đếm missing value từng cột.
- Loại cột flight.
- Tạo km và label.

Bài 2 - Categorical Encoding

- Index carrier và org.
- One-hot encode org.
- Giải thích sparse vector.

Bài 3 - Decision Tree

- Thử maxDepth = 3, 5, 10.
- So sánh Accuracy/F1/AUC.
- In toDebugString.

Bài 4 - Logistic Regression

- Tạo confusion matrix.
- Tính Precision, Recall, F1, AUC.

Bài 5 - SMS

- Tokenizer -> StopWords -> HashingTF -> IDF -> Logistic Regression.
- Báo cáo F1/AUC.

Bài 6 - Regression

- Dự đoán duration bằng km.
- Thêm org vào model.
- So sánh RMSE.

Bài 7 - Regularization

- So sánh Linear/Ridge/Lasso/Elastic Net.
- Đếm hệ số gần 0.

Bài 8 - Pipeline

- Tạo pipeline preprocessing + model.
- Giải thích vì sao fit trên train.

Bài 9 - Cross Validation

- Grid regParam và elasticNetParam.
- 3-fold CV.
- Đánh giá best model trên test.

Bài 10 - Ensemble

- So sánh Decision Tree/Random Forest/GBT bằng AUC và F1.

6. Câu hỏi ôn tập và yêu cầu nộp bài

Câu hỏi ôn tập

1. Regression và Classification khác nhau thế nào?
2. StringIndexer khác OneHotEncoder ở điểm nào?
3. Tại sao Spark ML cần cột features dạng vector?
4. Confusion matrix gồm những thành phần nào?
5. Khi nào Accuracy có thể gây hiểu nhầm?
6. TF-IDF giúp biểu diễn văn bản như thế nào?
7. RMSE và R^2 có ý nghĩa gì?
8. Lasso khác Ridge thế nào?
9. Data leakage xảy ra khi nào?
10. Pipeline hỗ trợ tránh leakage như thế nào?
11. Cross Validation dùng training set hay test set?
12. Random Forest khác Decision Tree đơn như thế nào?

Yêu cầu nộp bài

- Notebook đã chạy với output ở các phần được yêu cầu.
- Bảng so sánh metric giữa các mô hình.
- Kết quả bài tập them và mở rộng và nhận xét ngắn.
- Không chỉ chụp output; cần giải thích ý nghĩa metric và lựa chọn mô hình.
- Nếu thay đổi hyperparameter, ghi rõ cấu hình và seed.

Tổng kết

```
Data -> Preprocess -> Features -> Model -> Evaluate -> Tune -> Test
```

Chuỗi thao tác quan trọng cần nhớ: dữ liệu được đọc bằng DataFrame, biến phân loại được mã hóa, predictor được ghép vào features, preprocessing được fit trên training data, mô hình được đánh giá bằng metric phù hợp và hyperparameter được chọn bằng Cross Validation trước khi đánh giá cuối cùng trên test set.